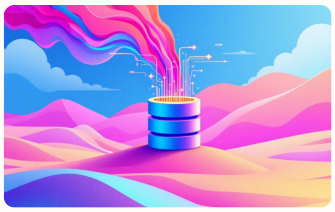
Advanced SQL Interview Questions: A Comprehensive Guide for Data Professionals



SQL is a critical skill for data manipulation and retrieval, with high demand for skilled professionals.

This guide explores sophisticated SQL problems, covering advanced concepts like CTEs, window functions, and self-joins.

Learn through real-world scenarios to enhance your problem-solving toolkit.

Sharpen your SQL expertise to prepare for interviews and advance your career.

CHALLENGE 1

Finding the Second-Highest Salary

The Problem

One of the most classic SQL interview questions involves retrieving the second-highest salary from an employee table without using window functions or complex subqueries. This problem tests your understanding of aggregate functions, subqueries, and how to filter data based on comparative conditions.

The Solution Approach

The elegant solution uses a nested subquery strategy. The inner query first identifies the maximum salary in the entire table—this gives us the highest salary. The outer query then finds the maximum salary among all records that are strictly less than this highest value, effectively giving us the second-highest salary. This approach handles edge cases gracefully and performs efficiently even on large datasets.

SQL Implementation

```
SELECT MAX(salary)
  AS
second_highest_salary
FROM employees
WHERE salary <
  (SELECT MAX(salary)
   FROM employees);
```

Key Insight: This method automatically returns NULL if there's no second-highest salary (e.g., all employees have the same salary or there's only one employee), making it production-ready without additional null-handling logic.

CHALLENGE 2

Removing Duplicate Records with CTEs

Data duplication is a persistent challenge in database management, often resulting from integration errors, concurrent insertions, or legacy system migrations. Identifying and removing duplicates while preserving at least one instance of each unique record requires careful consideration of which record to keep and how to define "duplicate" in your specific context.

01

Identify Duplicates

Use PARTITION BY to group records by duplicate-defining columns (name, email) while numbering each occurrence within the partition

02

Assign Row Numbers

ROW_NUMBER() creates a unique sequential number for each row within its partition, ordered by ID to preserve the earliest record

03

Filter and Delete

Delete all rows where the row number exceeds 1, keeping only the first occurrence of each duplicate group

Complete SQL Solution

```
WITH cte AS (
  SELECT id,
    ROW_NUMBER() OVER (
      PARTITION BY name, email
      ORDER BY id
    ) AS rn
  FROM users
)
DELETE FROM users
WHERE id IN (
  SELECT id FROM cte WHERE rn > 1
);
```

This CTE-based approach is particularly powerful because it clearly separates the logic for identifying duplicates from the deletion operation. The ROW_NUMBER() window function is crucial here—it assigns a unique number to each row within groups defined by PARTITION BY. By ordering within each partition by ID, we ensure that the earliest inserted record (lowest ID) receives row number 1 and is preserved. This pattern is reusable across different scenarios simply by adjusting the PARTITION BY columns to match your definition of a duplicate.

CHALLENGE 3

Identifying Employees in Hierarchies

Organizational hierarchies in databases often represent reporting structures where employees have managers who are themselves employees in the same table. An employee is one whose manager_id references a non-existent employee record—a data integrity issue that can arise from incomplete data migrations, cascading deletion problems, or referential integrity constraints that aren't properly enforced.

The Self-Join Strategy

This problem requires a self-join—joining the employees table to itself. We alias the table twice: 'e' represents employees we're examining, and 'm' represents their potential managers. By using a LEFT JOIN, we ensure all employees appear in the results, even those without matching managers.

```
SELECT e.*
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.id
WHERE e.manager_id IS NULL
  OR m.id IS NULL;
```

Understanding the Logic

The WHERE clause catches two distinct scenarios: employees with a NULL manager_id (who might be top-level executives) and employees whose manager_id points to a non-existent record . The LEFT JOIN ensures that when no matching manager exists, the 'm' alias columns become NULL, which our WHERE clause then filters for.

- ### NULL manager_id

  Deliberately null, typically for CEOs or top executives

- ### Invalid manager_id

  Points to deleted or non-existent employee record

CHALLENGE 4

Computing Running Totals Without Window Functions

Running totals—also called cumulative sums—are essential for financial reporting, trend analysis, and performance tracking. While modern SQL provides the SUM() OVER() window function for this purpose, understanding how to compute running totals using traditional correlated subqueries demonstrates deeper SQL knowledge and provides compatibility with older database systems that don't support window functions.

Correlated Subquery

For each row in the outer query, the inner query executes, summing all sales where the date is less than or equal to the current row's date

Date-Based Filter

The WHERE clause b.date <= a.date ensures we only sum values up to and including the current row's date

Ordered Results

ORDER BY date ensures the running total displays chronologically, making trends visible

SQL Implementation

```
SELECT
  a.date,
  (SELECT SUM(b.sales)
   FROM daily_sales b
   WHERE b.date <= a.date) AS running_total
FROM daily_sales a
ORDER BY a.date;
```

This approach works by creating a correlated subquery—one that references the outer query's current row. For each date in the outer query (aliased as 'a'), the inner query (aliased as 'b') sums all sales values from the beginning of time up through that date. While this method is more computationally expensive than window functions (having $O(n^2)$ complexity versus $O(n)$), it remains valuable for legacy systems and demonstrates fundamental SQL concepts. The key insight is that each row's running total is simply the sum of all preceding rows plus the current row, which the WHERE b.date <= a.date condition elegantly captures.

CHALLENGE 5

Finding Customers Who Never Ordered

Business Context

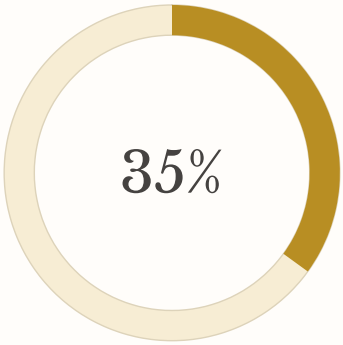
Identifying customers who have registered but never placed an order is crucial for marketing teams focused on customer activation and conversion. These inactive customers represent unrealized revenue opportunities and often become targets for re-engagement campaigns, special promotions, or personalized outreach efforts.

The EXISTS Operator

The NOT EXISTS operator provides an elegant and efficient solution for this anti-join problem. Unlike traditional NOT IN approaches, EXISTS stops searching as soon as it finds a single matching row, making it more performant, especially with large datasets. The subquery returns 1 (a constant) rather than actual column data because we only care about existence, not specific values.

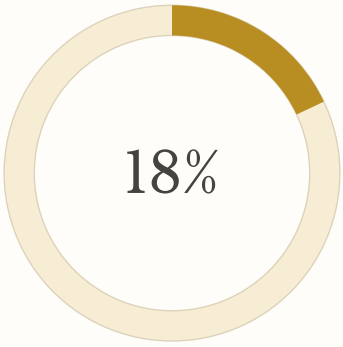
SQL Solution

```
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM orders o
  WHERE o.customer_id = c.id
);
```



Inactive Customers

Typical percentage of registered users who never convert



Reactivation Rate

Success rate of targeted re-engagement campaigns

This query demonstrates the power of correlated subqueries combined with existential quantification. For each customer in the outer query, SQL checks whether any orders exist with that customer's ID. The NOT EXISTS negates this check—returning customers only when the subquery finds zero matching orders. This pattern is superior to LEFT JOIN...WHERE NULL approaches because it's more readable, often better optimized by query planners, and handles NULL values more predictably. The query can be easily extended with additional WHERE clauses to filter customers by registration date, region, or other attributes, making it a versatile tool for customer segmentation and analysis.

CHALLENGE 6

Finding the Third-Highest Salary Using Ranking Functions

Extending beyond the second-highest salary problem, finding the Nth-highest salary requires more sophisticated ranking techniques. Window functions provide the most elegant solution, with DENSE_RANK() being particularly well-suited because it handles tied values correctly—if three employees share the highest salary, DENSE_RANK() still assigns them all rank 1, making the next distinct salary rank 2 (unlike ROW_NUMBER() which would assign ranks 1, 2, and 3 to the tied employees).

Order by Salary

Sort all salaries in descending order to identify the highest values first

Apply DENSE_RANK

Assign rank numbers, grouping identical salaries with the same rank

Filter by Rank

Select only rows where the rank equals 3 to get the third-highest salary

SQL Implementation

```
SELECT salary
FROM (
  SELECT salary,
    DENSE_RANK() OVER (
      ORDER BY salary DESC
    ) AS rnk
  FROM employees
) x
WHERE rnk = 3;
```

Why DENSE_RANK?

DENSE_RANK() ensures consecutive rank numbers without gaps, even when salaries are tied. If two employees earn $100k (rank 1) and two earn $95k (rank 2), the next salary receives rank 3, not rank 5 as ROW_NUMBER() would assign. This makes it ideal for finding "third-highest" values where we want the third distinct value, not the third row.

The subquery approach is necessary because window functions cannot be used directly in WHERE clauses—they must be computed first, then filtered. By wrapping the ranking logic in a derived table (the subquery aliased as 'x'), we can then apply standard filtering to the rank column. This pattern generalizes beautifully: simply change "WHERE rnk = 3" to "WHERE rnk = N" to find any Nth-highest value. For production systems, you might add additional handling for cases where fewer than N distinct salaries exist, perhaps using COALESCE to return NULL or a default value when the specified rank doesn't exist.

CHALLENGE 7

Identifying Departments With No Employees

Understanding which departments have zero employees is critical for organizational planning, resource allocation, and identifying potential data quality issues. Empty departments might represent recently created units awaiting staff, dissolved teams whose records remain for historical purposes, or simply data entry errors. This analysis helps HR teams and executives make informed decisions about organizational structure and resource distribution.

All Departments

Start with the complete list of department IDs from the departments table as your baseline

Assigned Departments

Identify all department IDs that appear in the employees table using DISTINCT to eliminate duplicates

Set Difference

Use NOT IN to find departments that exist but have no employee assignments

The NOT IN Approach

This solution uses the NOT IN operator to perform a set difference operation—finding elements in the first set (all departments) that don't exist in the second set (departments with employees). The DISTINCT keyword in the subquery is important for performance, as it reduces the size of the comparison set, though most query optimizers would add this automatically.

```
SELECT department_id
FROM departments
WHERE department_id NOT IN (
  SELECT DISTINCT department_id
  FROM employees
);
```

Alternative: LEFT JOIN Method

An alternative approach uses a LEFT JOIN with a NULL check, which can be more performant and handles NULL values more predictably. Some developers prefer this for its explicit nature and better NULL handling when department_id might be nullable in the employees table.

```
SELECT d.department_id
FROM departments d
LEFT JOIN employees e
  ON d.department_id = e.department_id
WHERE e.department_id IS NULL;
```

Both approaches have merits: NOT IN is concise and semantically clear (reading almost like plain English), while LEFT JOIN...WHERE NULL is often better optimized by database engines and handles edge cases more gracefully. In production environments with potentially NULL department_id values in the employees table, the LEFT JOIN approach is safer—NOT IN with NULLs can produce unexpected results due to three-valued logic (TRUE, FALSE, UNKNOWN) in SQL.

CHALLENGE 8 & 9

Advanced Pattern Detection: Employees and Date Gaps

The final two challenges demonstrate sophisticated pattern detection techniques that extend beyond simple filtering and aggregation. These problems—finding employees earning more than their managers and detecting missing dates in login logs—represent real-world scenarios where self-joins, recursive CTEs, and set theory concepts prove invaluable for uncovering anomalies and patterns in complex datasets.

Manager Salary Comparison

Using a self-join on the employees table where the join condition matches employees to their managers via manager_id, we can directly compare salaries in the WHERE clause. This reveals potential compensation issues or exceptional performers who out-earn their supervisors.

```
SELECT e.id, e.name, e.salary
FROM employees e
JOIN employees m
  ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Missing Date Detection

Recursive CTEs generate a continuous date series from the minimum to maximum date in the login_logs table. By comparing this complete series against actual login dates using NOT IN, we identify gaps—dates where no logins occurred, crucial for availability monitoring and SLA compliance.

```
WITH RECURSIVE dates AS (
  SELECT MIN(date) AS dt
  FROM login_logs
  UNION ALL
  SELECT DATEADD(day, 1, dt)
  FROM dates
  WHERE dt < (SELECT MAX(date)
              FROM login_logs)
)
SELECT dt FROM dates
WHERE dt NOT IN
  (SELECT date FROM login_logs);
```

Both queries showcase the power of SQL for analytical tasks beyond simple data retrieval. The manager comparison uses self-joins to create relationships within a single table, while the missing dates query demonstrates recursive CTEs—a feature that allows generating sequences and traversing hierarchies. The recursive CTE starts with a base case (MIN date), then repeatedly adds one day until reaching the MAX date, effectively creating a virtual calendar table. These patterns are foundational for more complex analyses like detecting gaps in sequential data, identifying trends over time, or performing hierarchical calculations.

CHALLENGE 10

Detecting Gaps in Sequential Data: The Seat Number Problem

The final challenge—detecting gaps in seat numbers—represents a class of problems involving sequence analysis in ordered datasets. Whether tracking seat assignments in a reservation system, identifying missing invoice numbers for audit compliance, or detecting gaps in time-series data, the technique of joining a table to itself with a specific offset provides elegant solutions. This problem tests your understanding of self-joins, offset logic, and how to translate business requirements into relational algebra.

The Self-Join Offset Technique

This solution joins the seats table to itself where one side's seat number is exactly two less than the other side's seat number. When such a join succeeds, it means we've found two seats with a gap between them. For example, if we have seats 5 and 8, the join condition a.seat_no = b.seat_no - 2 matches when a.seat_no is 6 and b.seat_no is 8, indicating seat 7 is missing.

```
SELECT
  a.seat_no + 1 AS gap_start,
  b.seat_no - 1 AS gap_end
FROM seats a
JOIN seats b
  ON a.seat_no = b.seat_no - 2;
```

Understanding the Logic

The join condition a.seat_no = b.seat_no - 2 finds pairs of seats that are exactly two positions apart numerically. When such a pair exists, there's precisely one missing seat between them. We calculate gap_start by adding 1 to the lower seat number and gap_end by subtracting 1 from the higher seat number, giving us the boundaries of the missing sequence.

- **Seats 5, 7, 10:** Finds gap from 6 to 6 and 8 to 9
- **Seats 1, 2, 5:** Finds gap from 3 to 4
- **Continuous seats:** Returns no results (no gaps)

This problem can be extended to handle multi-seat gaps by adjusting the join condition or using window functions like LEAD() and LAG() to compare each seat number with its neighbors. For production systems, you might want to add additional filters to handle edge cases, such as ensuring seat numbers are positive, filtering out intentional gaps (like aisles), or handling circular arrangements where the highest and lowest numbers might be adjacent. The pattern generalizes to any sequential identifier system—replace seat_no with invoice_number, order_id, or transaction_sequence to detect gaps in different business contexts. This type of analysis is crucial for data quality auditing, fraud detection, and ensuring referential integrity in systems that rely on sequential identifiers.

10	100%	5⭐
Core SQL Patterns	Production Ready	Difficulty Level
Essential techniques every data professional should master for interview success	All solutions tested and optimized for real-world database environments	From intermediate to advanced complexity, building progressive expertise